Question 1:
Scope defines where variables can be accessed. JavaScript has global, function, and block scope. Hoisting is JavaScript's behavior of moving declarations to the top of their scope. Closures allow a function to access variables from its outer scope even after execution. These concepts affect memory usage and variable accessibility in large applications.

Question 2:
ES6 introduced features like Arrow Functions, Destructuring, and Async/Await. Arrow functions simplify syntax and preserve context. Destructuring extracts values from arrays/objects easily. Async/Await simplifies asynchronous code, making it readable and maintainable.

Question 3:
A real-world application is an e-commerce website using JavaScript on both frontend and backend. Frontend uses React for UI, backend uses Node.js. JavaScript handles real-time data updates, authentication, and API communication efficiently.

Question 4:
Debugging large apps requires logging, browser dev tools, breakpoints, and performance profiling. Tools like Chrome DevTools, ESLint, and monitoring tools help detect issues. Best practices include modular code, proper error handling, and testing.

Question 5:
Build tools like Webpack and Vite bundle code efficiently. Package managers like npm manage dependencies. These tools improve performance, automate tasks, and optimize deployment.